

Day 40



Classes and Objects in JavaScript:

What is a Class?

A class is a blueprint or template for creating objects. It describes what properties (data) and what methods (functions) the objects will have.

Think of a class as a “design” and objects as the “real things” created from that design.

What is an Object?

An object is an actual thing created from a class. Each object has its own copy of properties, but shares methods defined in the class.

Basic Syntax of a Class

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  greet() {  
    console.log("Hello, I am " + this.name);  
  }  
}  
  
let p1 = new Person("Aditya", 21);  
p1.greet();
```

Example: creating a simple class with the methods.

```
JS main.js > RailwayForm  
1 //Class name is RailwayForm, methods name are submit and cancel  
2 class RailwayForm {  
3   submit(){  
4     console.log("Form submitted");  
5   }  
6   cancel(){  
7     console.log("Form cancelled");  
8   }  
9 }
```

Example: creating two instances of class

```
JS main.js > ...
1 //Class name is RailwayForm, methods name are submit and cancel
2 class RailwayForm {
3     submit(){
4         console.log("Form submitted");
5     }
6     cancel(){
7         console.log("Form cancelled");
8     }
9 }
10
11 // Create two instances of RailwayForm
12 let Aditya = new RailwayForm();
13 let Utsav = new RailwayForm();
14
15 Aditya.submit(); // Output: Form submitted
16 Utsav.cancel(); // Output: Form cancelled
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day31-40\Day40> node main.js
Form submitted
Form cancelled

Constructors in JavaScript:

What is a Constructor?

A constructor is a special function inside a class that automatically runs when you create an object. Its job is to set the initial values of the object.

In simple words:

A constructor prepares the object when it is created.

Constructor Syntax

```
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
}
```

- constructor() → special function
- this.name / this.age → properties added to the object

Creating an Object

```
let p1 = new Person("Aditya", 21);

console.log(p1.name);

console.log(p1.age);
```

When new Person("Aditya", 21) runs:

- constructor is automatically called
- it sets:
 - this.name = "Aditya"
 - this.age = 21

Example: Adding Methods Along with Constructor

```
18 class Car {
19   constructor(model, year) {
20     this.model = model;
21     this.year = year;
22   }
23   show() {
24     console.log(this.model, this.year);
25   }
26 }
27 let c = new Car("BMW", 2023);
28 c.show();
29
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day31-40\Day40> node main.js
BMW 2023

Example: constructor Without Parameters

```
30 class Example {
31   constructor() {
32     this.value = "Hello";
33   }
34 }
35 let e = new Example();
36 console.log(e.value);
37
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\JavaScript\Day31-40\Day40> node main.js
Hello

Also, **If You Don't Write a Constructor** - JavaScript automatically creates an empty one:

```
constructor() {}
```

So, the class still works.

Short Summary

- Constructor = a special function that prepares an object.
- It runs automatically when using new.
- It sets properties using this.
- Child classes use super() to call parent constructor.
- If you don't write one, JS creates a default constructor.

--The End--